

MVC – обобщение за изпитване

ASP = Active Server Page – интернет страница с HTML и скриптове, за уеб приложения.

ASP .Net Core е крос-платформа (работи на Windows и Linux), с висока производителност, отворен код за създаване на модерни, облачно-базирани, свързани с интернет приложения.

SDK = Software Development Kit – колекция от софтуерни инструменти в един инсталиран пакет.

Runtime = Run only – само стартира проект, но не може да го създаде.

Core – ядро - съдържа всичко в себе си (може да го проектира и стартира с Runtime).

* Empty – празен, ние трябва да пишем всичко

* MVC – Model-View-Controller – генерира най-много авто-код. Съдържа всичко наготово генерирано – правила сме.

* Razor = ножица, резачка - изрезка от сходен код, шаблон по образец.

HTML = HyperText Markup Language – език за маркиране на текст с хиперлинкове за уеб сайтове.

http = HyperText Transfer Protocol – протокол за трансфер на HTML страници в интернет.

https = http + s, където S = Secured, защитен http, който е важен за Login с пароли, за да не се покажат в адрес-бара (навигационната лента) на браузъра.

.NetCore app – започва като конзолно приложение (терминала) и по-късно се развива като web.

IDE = Integrated Development Environment = Интегрирана среда за разработка на софтуер / СРЕДАТА, т.е. в нашия случай е Visual Studio 2022 / VS.

Framework = колекция от готови използваеми софтуерни компоненти, които облекчават и ускоряват работата на програмиста.

Всички упражнения в училище досега са правени на .NET Framework 5.0, който съдържа Startup.cs с main – метода (входната точка на програмата). От .net 6.0 нататък, той отпада.

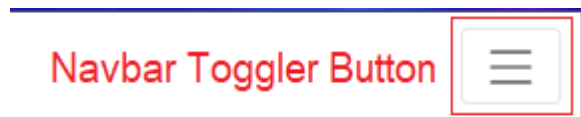
.cshtml = C# + html - комбинация от 2 езика, блендирани (смесени) в 1 страница.

BOOTSTRAP = HTML + CSS + JS библиотека, автомащабиране.

През 2015 г се въвежда новият стандарт HTML5, който решава проблема с автоматичното преоразмеряване на екраните заради GSM-те. Това се случва, благодарение на

1) 1 ред код: `<meta name="viewport" content="width=device-width, initial-scale=1.0" />`

2) **Toggler – бутон за свиване на менюто**



InProcess има много по-голяма пропускателна способност от OutOfProcess.

Имаме 2 локални сървъра: вътрешен (Kestrel) + външен (iis express). Когато разработваме проект (т.е. development) искаме да имаме по-голяма видимост, затова използваме и двата сървъра (вътрешният междинен ни помага за контрол). Когато, обаче, публикуваме официално сайта и

всичко е готово и без грешки (InProduction) искаме бързина за потребителите, затова средният слой (сървър) става излишен и използваме само външния.

OutOfProcess = 2 сървъра (вътрешен Kestrel + външен IIS Express), затова е по-бавен, Development

InProcess = 1 сървър (външен IISExpress) – много по-бърз.

InProduction използваме iis

IIS = Information Internet Serrver

Възможните стадии на средите (Environment) при разработка на проект са 3:

Development – по време на разработка

Staging – финални изпитания

Production – официално публикуване за ползване от потребителите в интернет.

Ако не е написана среда, се използва тази, която е заложена по подразбиране в Windows – Production.

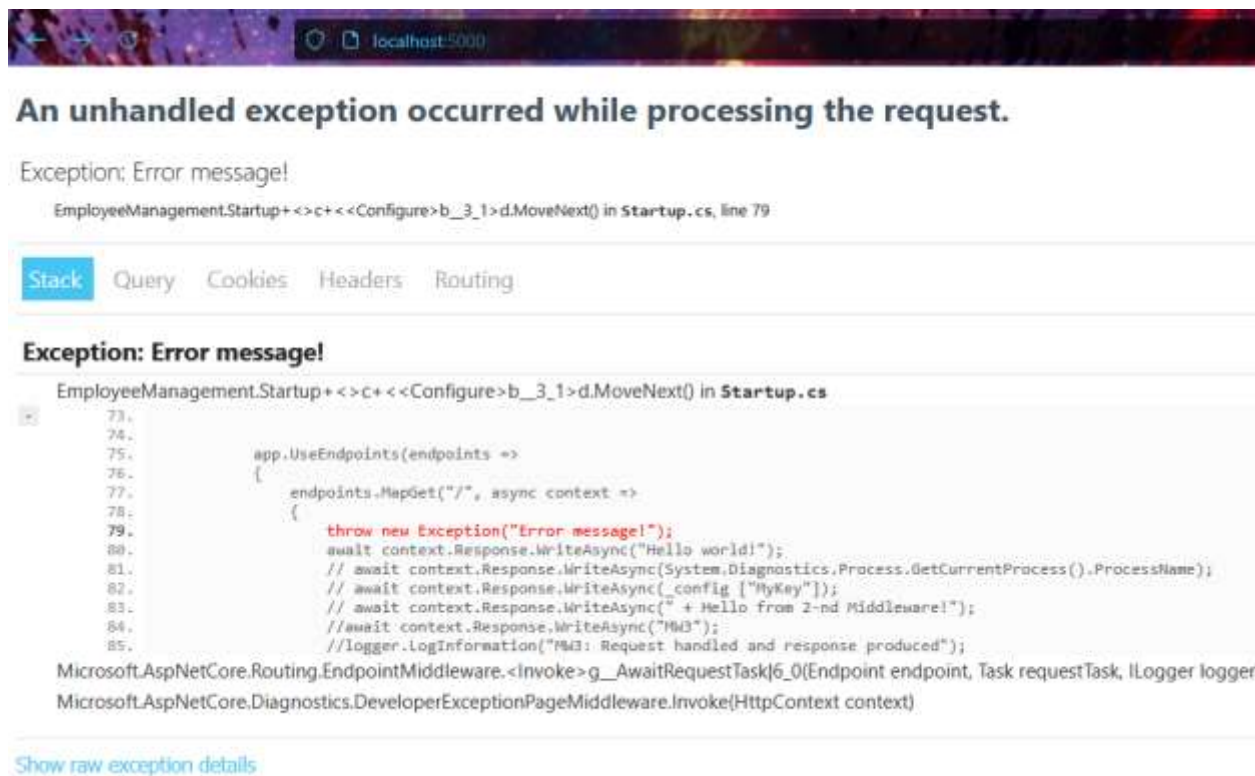
CMD = Command Prompt – конзолата / терминала, в който се въвеждат команди (черния прозорец)

CLI = Command Line Interpreter – CMD на Visual Studio се нарича така.

API – Application User Interface – интерфейсът за потребителите

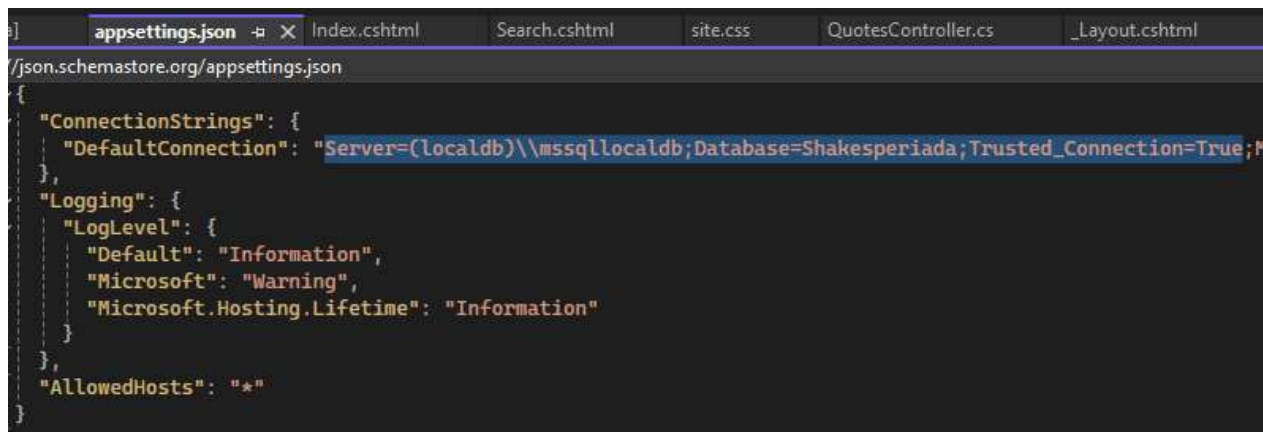
GUI – Graphic User Interface

Developer Exception Page – страница, която носи информация за грешки. По време на разработка (Development) е добре да я има, но когато се пусне на потребителите (Production) не, т.к. някой може да злоупотреби, а и такива неграфични страници плашат аудиторията.

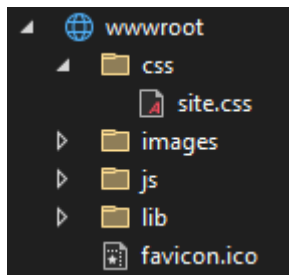


Настройките за връзката със сървъра и БД се правят в appsettings.json

Server= ;
Database= ;
Trusted_Connection=True;



В wwwroot се намират CSS – Cascading Style Sheets със стиловете за украсяване на фон, цветове, текст, големина на буквите, отстояния... Там се намира и папката с картинки и favicon.ico – иконката в таба на браузъра.



DRY = Don't Repeat Yourself – основна концепция в програмирането, при която често повтарящият се код се отделя на друго място, а при необходимост само се извиква (чрез методи или други начини).

В нашия случай, скелетът на HTML е отделен в папка Views -> Shared -> _Layout.cshtml

Models Views and Controllers (MVC)

- Model = classes (objects)
- View = web page (Razor HTML)
- Controller = connects models, business logic and web pages.

Модел-Изглед-Контролер (Model-View-Controller или MVC) е архитектурен [шаблон за дизайн](#) в програмирането, основан на разделянето на бизнес логиката от графичния интерфейс и данните в дадено приложение.

MVC = Model + View + Controller

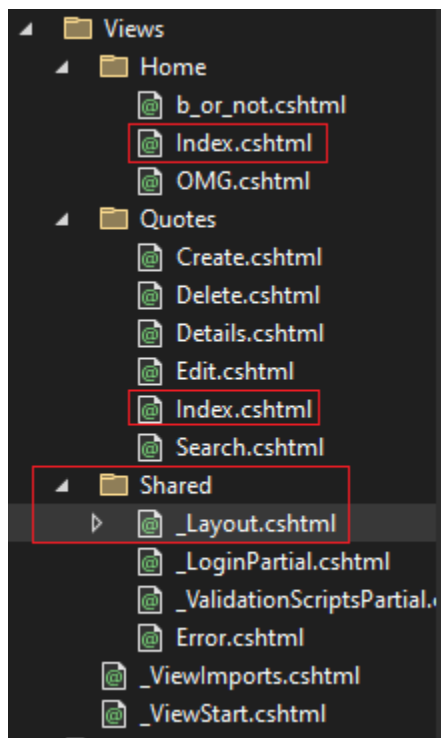
Model = логиката = това са свойствата settings за БД (int ID, string Name... {get; set;})

View = видимите страници в интернет

Controller = действието (action) – обикновено е само return View (да покаже страницата).

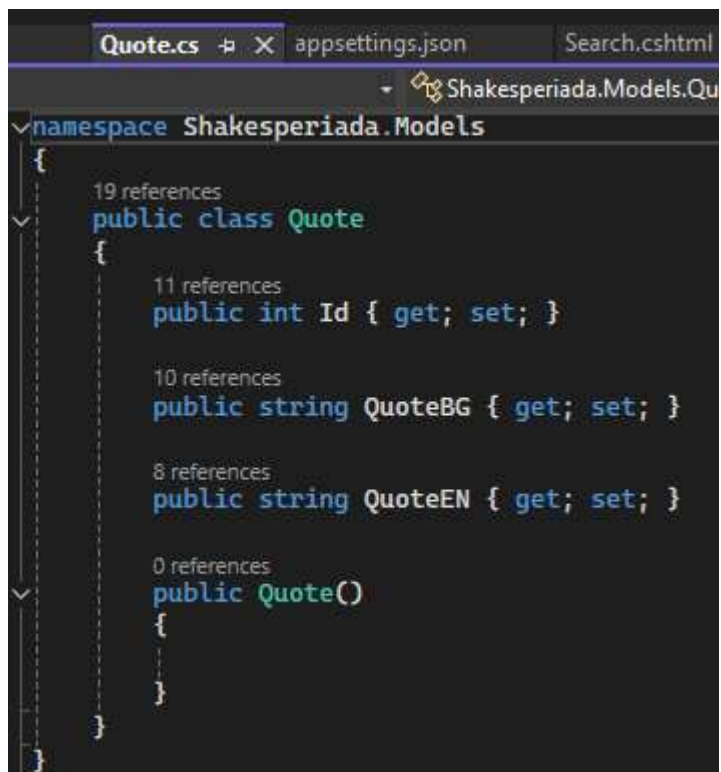
Винаги правим първо модела, за да може той да бъде извикан от контролера при създаването на БД. Ако моделът е празен, контролерът ще хвърля грешки.

Views / Изгледи = това са .cshtml страниците, които потребителите виждат в браузъра. Намират се в папка Views. По подразбиране се зарежда Home/Index.cshtml. Това означава, че ще потърси HomeController и в него action Index.



В папка Shared, както показва името ѝ, са общодостъпните за останалите части на проекта парчета код.

Ето как изглежда един модел, който държи логиката. Първо се създава модела, за да може после да бъде извикан от контролера:



Моделът наподобява обекти от околния свят, чрез които създаваме подобие като за БД.

Ето как изглежда друг модел:

```
namespace EmployeeManagement.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Email { get; set; }
        public string Department { get; set; }
    }
}
```

Откъс от упражнение на групата на г-жа Стойчева:

2. Добавяне на модела (Model)

1. В папката **Models** създайте клас **Product.cs**:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Recipient { get; set; }
    public string Supplier { get; set; }
    public decimal Price { get; set; }
    public string Type { get; set; } // Optional: "Necessary" or
    "Unnecessary"
}
```

Т.е. там задаваме атрибутите / свойствата на колоните от таблица на БД!!!!

Контролерът извиква действието (action), обикновено е return View, т.е. да се покаже страницата.

```
0 references
public IActionResult OMG()
{
    return View();
}

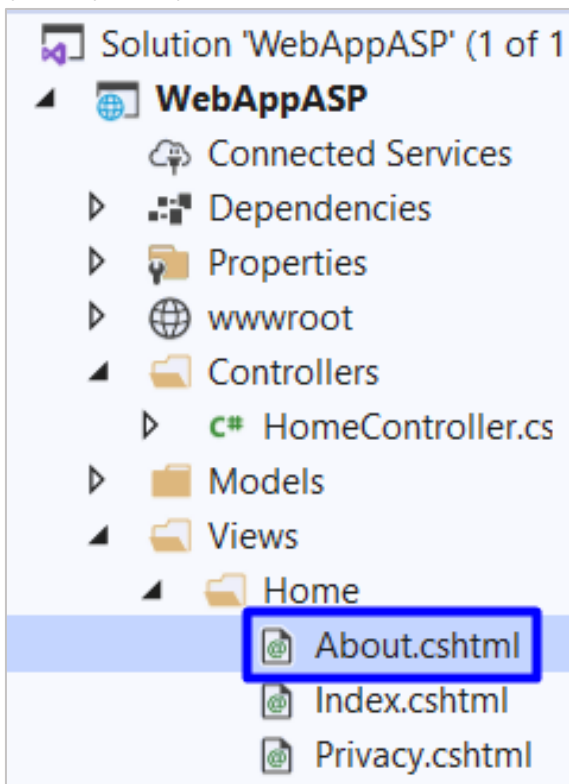
0 references
public IActionResult Index()
{
    return View();
}

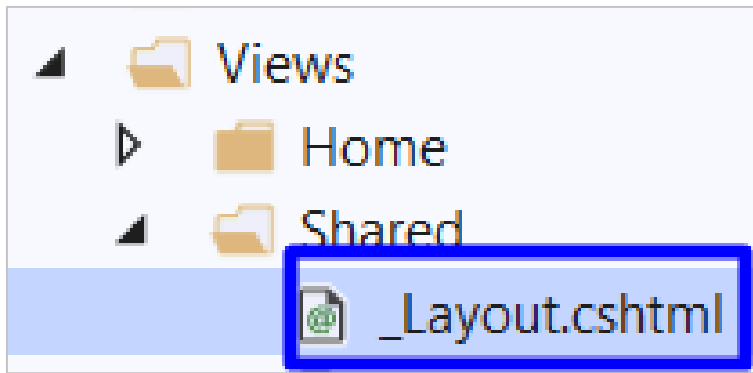
0 references
public IActionResult b_or_not()
{
    return View();
}
```

Да анализираме адреси – упражнение.

- 1) Ако в браузъра заредим URL/Home/About – отива в HomeController, намира action About(), вижда return View(); отива в папка Views -> Home -> About.cshtml и показва нея.

\Views\Home\About.cshtml





Обикновено, потребителите стигат до този адрес от менюто за навигация Navbar, който е зададен в _Layout.

```
<div class="navbar-collapse collapse d-sm-inline-flex flex-sm-row-reverse">
  <ul class="navbar-nav flex-grow-1">
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
    </li>
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
    </li>
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="About">About</a>
    </li>
  </ul>
</div>
```

- Returns the view in **Views/{controller}/{action}.cshtml**

Примери:

/Home/OMG ->

търси HomeController, action OMG, вижда return View() ->
отива във Views -> Home -> OMG.cshtml

/Quotes

щом е зададена само 1 директория за папка, това е QuotesController, action Index. Никъде в
браузъра няма да бъде прочетен динамичен index.cshtml

localhost:xxxx -> ако не пише нищо, по подразбиране зарежда Home->index.cshtml

/Quotes/Search

QuotesController, action Search -> папка Views/Quotes/Search.cshtml

/Employees/Edit

EmployeesController, action Edit -> папка Views/Employees/Edit.cshtml

/Towns

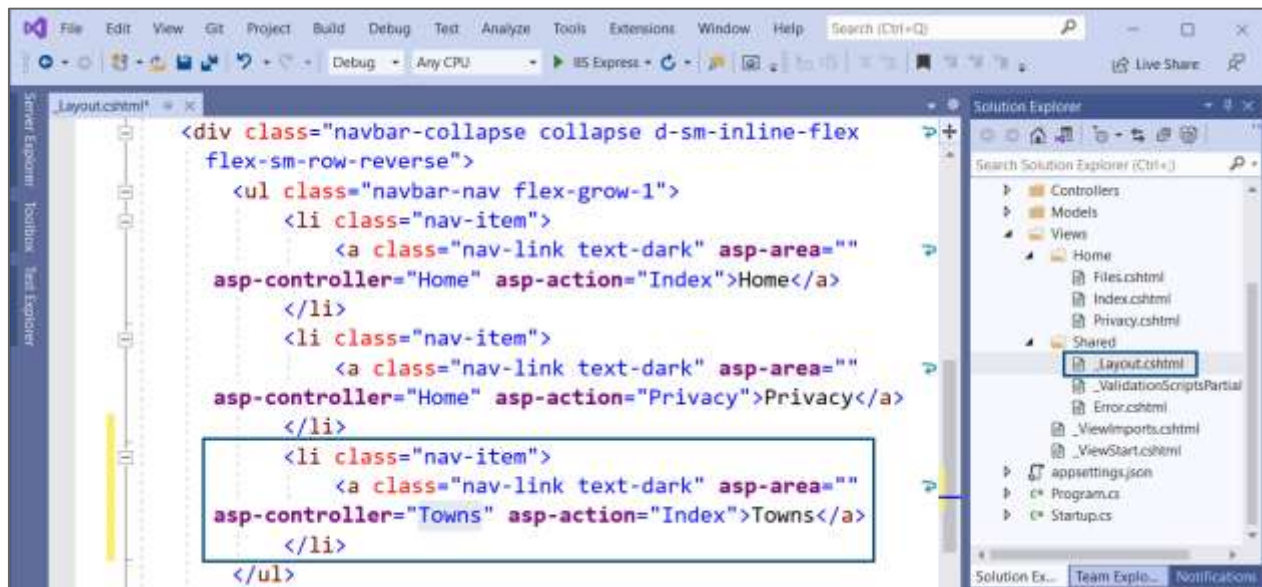
TownController, action index -> папка View/Towns/index.cshtml

CRUD – операции = Create, Read, Update, Delete.

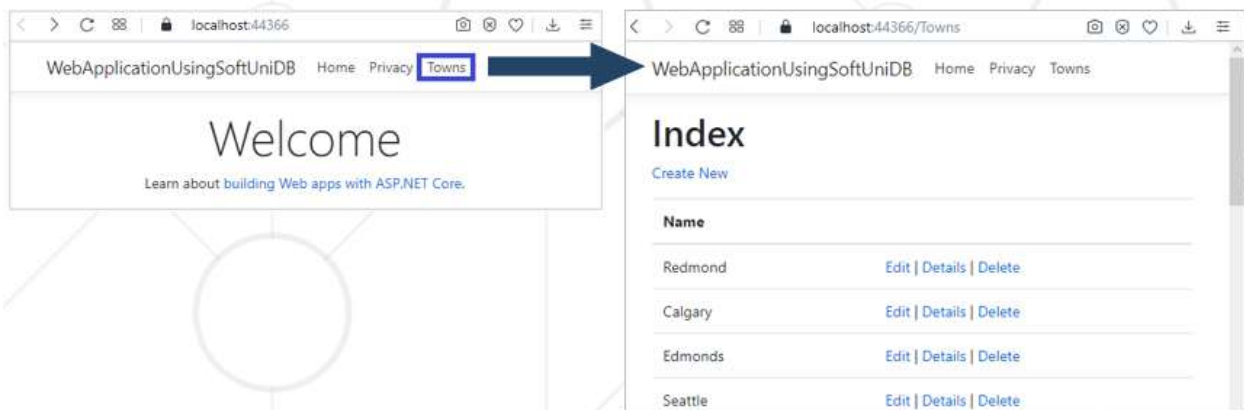
Index Create New	
Name	
Redmond	Edit Details Delete
Calgary	Edit Details Delete
Edmonds	Edit Details Delete
Seattle	Edit Details Delete

/Towns/Delete

TownsController, action Delete ->Views/Towns/Delete.cshtml



```
<div class="navbar-collapse collapse d-sm-inline-flex flex-sm-row-reverse">
  <ul class="navbar-nav flex-grow-1">
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area=""
        asp-controller="Home" asp-action="Index">Home</a>
    </li>
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area=""
        asp-controller="Home" asp-action="Privacy">Privacy</a>
    </li>
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area=""
        asp-controller="Towns" asp-action="Index">Towns</a>
    </li>
  </ul>
</div>
```



Generated CRUD Controllers



- We can retrieve an **entire collection**, using the **DbContext**:

```
// GET: /Towns/Index
public IActionResult Index()
{
    return View(this._context.Towns.ToList());
}
```

- Generated route: **"/Towns/Index"**

Generated CRUD Controllers (2)



- **Asynchronous** version of the previous controller:

```
// GET: /Towns/Index
public async Task<IActionResult> Index()
{
    return View(await _context.Towns.ToListAsync());
}
```

- Asynchronous code is **more efficient** (load faster)

Текст label = етикет, по същия начин както във формите.
поле input = въвеждане на текст, пак като при формите.

```

<div>
  <label asp-for="Name"></label>
  <input asp-for="Name"/>
  <span asp-validation-for="Name"></span>
</div>

```

Валидация validation – проверка за брой букви, главни, малки, @, специални символи и числа.

Има два начина за създаване на БД за система като тази DAO и ORM:

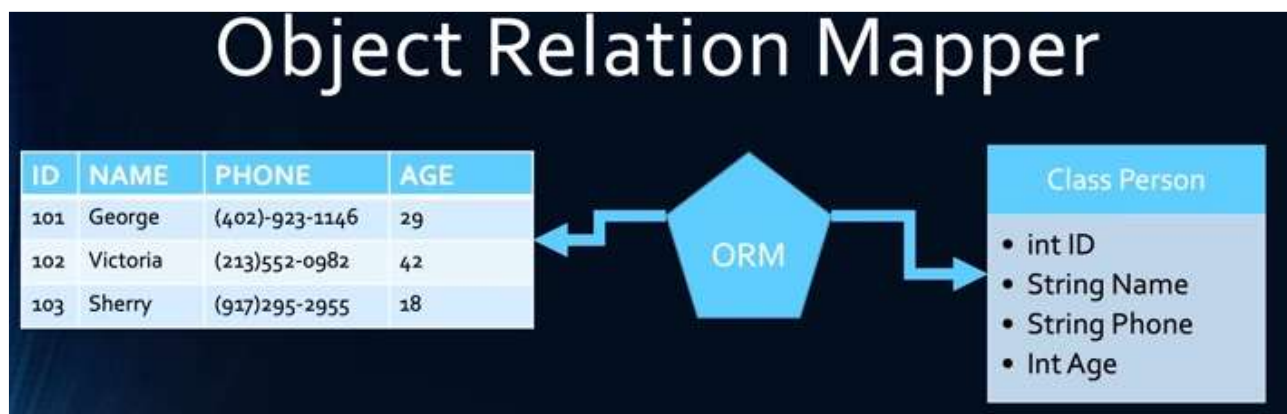
DAO (data access object): - първо БД, после приложението – DB first

*традиционен, старомоден с ръчно въвеждане и все още предпочитан от компаниите и програмистите, заради по-голямата видимост какво се случва в БД и съответно по-големия контрол;

ORM (object relation mapper): - първо приложението, после БД – code first

* предпочитан от програмистите, когато искат да спестят време; автоматично генерира таблици без да се налага да пишат никакви SQL-твърдения/отчети; автоматично генерира таблици въз основа на дефинирани класове в приложенията; БД се обновява чрез миграциите; подходящи за **прости** приложения.

Entity – името на ORM на Майкрософт, който използваме в момента;



Миграциите са езикът с който Visual Studio създава ORM бази данни. Първата миграция се нарича initialmigration стандартно, за да я разпознаваме. Първата команда е:

Add-migration initialmigration

Но тя не създава БД. Тя само запазва командата. БД се създава едва след втората команда:

update-database

БД са тежки, обновяват се бавно и при тях всичко е след apply, refresh, update.